

课程笔记

Codex 初步：系统提示词注入与 Context Engineering

五道口纳什 & AI

2026 年 3 月 28 日

Codex 初步 系统提示词注入 Context 管理 Skills / MCP 暴露

视频作者/频道：五道口纳什

发布日期：2025

视频时长：约 8 分钟

视频链接：未填写

目录

1 分析对象与方法	2
1.1 从 Session 文件逆向分析 Codex	2
1.2 本章小结	2
2 Codex 的消息历史结构	2
2.1 四条预注入消息	2
2.2 各条消息的详细内容	2
2.2.1 第一条: System Instruction	2
2.2.2 第二条: Developer 权限注入	2
2.2.3 第三条: User 侧注入	3
2.2.4 第四条: Developer 协作模式	3
2.3 本章小结	3
3 消息历史的运行时管理	3
3.1 多轮交互的 Context 管理	3
3.2 Skills 的渐进式加载	4
3.3 本章小结	4
4 如何读取 Codex 的 System Prompt	4
4.1 读取方法	4
4.2 本章小结	4
5 总结与延伸	4
5.1 核心要点回顾	4
5.2 与下期的衔接	5

1 分析对象与方法

1.1 从 Session 文件逆向分析 Codex

本期通过逆向分析 Codex 的会话历史文件，揭示其 Context Engineering 的设计。当用户在 Codex CLI 中发送指令时，Codex 会将完整的对话历史保存为本地 JSONL 文件。通过分析这些文件，可以还原 Codex 内部的消息管理机制。

合法的提示词注入

本期讨论的“提示词注入”是指 Codex、Claude Code、OpenAI Codex 等现代 Agent 工具在内部对消息历史的管理——即用户在 TUI 界面发送第一条指令之前，工具已经预先植入了大量的系统指令。这是合法的、设计上的注入，不同于恶意的 Prompt Injection 攻击。

1.2 本章小结

通过分析 Codex 的 JSONL 会话文件，可以完整还原其消息历史的管理方式，理解 Modern Agent 的 Context Engineering 设计。

2 Codex 的消息历史结构

2.1 四条预注入消息

在用户发出真实请求之前，Codex 已经预先植入了至少四条消息：

Codex 预注入的消息结构

1. **System 消息**：基础的系统指令（System Prompt）
2. **Developer 消息（第一条）**：权限注入，定义 Codex 的操作权限边界
3. **User 消息（注入）**：包含 agents.md 的内容、Skills 摘要、环境上下文
4. **Developer 消息（第二条）**：定义协作模式，描述 Codex 应如何与用户交互

2.2 各条消息的详细内容

2.2.1 第一条：System Instruction

System 消息是最顶层的指令，定义了 Codex 的基本行为规范和能力边界。这是一个标准的 System Prompt。

2.2.2 第二条：Developer 权限注入

Developer 消息用于注入权限相关的配置，定义 Codex 在文件操作、命令执行等方面的权限边界。

2.2.3 第三条：User 侧注入

User 消息的丰富内容

第三条消息虽然标记为 User 角色，但实际是 Codex 工具自动注入的，包含三大部分：

- **agents.md**：系统级或用户级的 Agent 配置文件，从 `.codex` 目录或代码工程根目录加载
- **Skills 摘要**：列出所有可用的 Skills 及其简短描述（渐进式加载的索引）
- **环境上下文**：当前工作目录、操作系统信息、当前日期、时区等运行时信息

关于 Skills 的注册，其结构包括：

- 哪些 Skills 是 available 的
- 每个 Skill 的简短描述
- 官方默认的两个内置 Skill：Skill Creator 和 Skill Installer
- 如何使用各个 Skill 的说明

2.2.4 第四条：Developer 协作模式

第四条 Developer 消息定义了 Codex 应如何与用户进行协作——交互风格、响应方式、Turn 管理等。它还维护了一个 Turn 标记/上下文，用于追踪对话轮次。

2.3 本章小结

Codex 在用户交互开始前就植入了 4 条消息，覆盖系统指令、权限、Agent 配置 +Skills+ 环境、协作模式，构建了完整的 Agent Runtime 上下文。

3 消息历史的运行时管理

3.1 多轮交互的 Context 管理

在预注入的 4 条消息之后，Codex 进入真正的用户-Agent 多轮交互阶段：

1. 用户发出一个真实的 Query
2. Agent Loop 开始执行多轮（multi-turn）交互
3. 交互过程中包含工具调用（Function Call）及其返回结果
4. 一个 Turn 结束
5. 用户发出新的 Query，开始下一个 Turn

Compact 指令的作用

当消息历史积累过多时，可以在 Codex CLI 中执行 `compact` 指令。Compact 压缩的是多轮对话部分的内容，但会保留预注入的系统消息。这是 Context Engineering 中管理上下文窗口的关键手段。

3.2 Skills 的渐进式加载

Skills 的两阶段加载机制

Skills 采用渐进式加载 (Progressive Loading) 策略：

1. **阶段一 (注册)**：在预注入的 User 消息中，只暴露每个 Skill 的名称和简短描述
2. **阶段二 (加载)**：当远端模型决定使用某个具体 Skill 时，通过 Function Call 执行 `cat` 命令，将该 Skill 对应的 `skills.md` 文件完整内容作为 Function Output 加载到 Context 中

这种设计避免了一次性加载所有 Skills 导致的 Context 浪费。

3.3 本章小结

Codex 的消息历史在运行时持续增长，通过 `compact` 指令进行上下文压缩。Skills 采用渐进式加载，先暴露摘要，按需加载完整内容，是一种精巧的 Context 管理策略。

4 如何读取 Codex 的 System Prompt

4.1 读取方法

对于希望深入了解 Codex 内部指令的用户，可以：

1. 找到 Codex 保存的 Session 文件（本机的 JSONL 文件）
2. 提取其中的 System、Developer、User 各条消息内容
3. 内容本身是标准的 Markdown 格式，可以用在线 Markdown 渲染工具查看
4. 可以使用翻译工具对英文内容进行逐层翻译阅读

Session 文件的位置

Codex 的 Session 文件保存在本机特定目录下，具体路径取决于操作系统和 Codex 版本。每次会话对应一个 JSONL 文件，文件中按时间顺序记录了完整的消息历史。

4.2 本章小结

通过读取 Codex 的 JSONL Session 文件，可以获取完整的 System Prompt、Developer 消息和所有注入内容，是理解 Modern Agent 内部设计的有效途径。

5 总结与延伸

5.1 核心要点回顾

本期通过逆向分析 Codex 的会话文件，揭示了 Modern Coding Agent 的 Context Engineering 设计：

- Codex 在用户交互前预注入 4 条消息 (System → Developer → User → Developer)

- User 注入消息包含 agents.md、Skills 摘要、环境上下文
- Skills 采用渐进式加载：先暴露索引，按需加载完整内容
- 多轮对话通过 compact 指令进行上下文压缩
- 整个设计的核心是消息历史的维护与管理

Context Engineering 的本质

无论是 Codex、Claude Code 还是 OpenAI Codex，所有 Modern AI Agent 的核心工作都可以归结为一件事：**Context Engineering**——精心设计和管理送给大模型 API 的消息列表 (Message List)，包括预注入的指令、Skills 的暴露方式、工具的注册、对话历史的压缩等。

5.2 与下期的衔接

本期为下一期介绍 Superpowers Skills 做了技术铺垫。理解了 Codex 如何管理消息历史、如何渐进式加载 Skills 之后，就能更好地理解 Superpowers 这类复杂 Skills 的设计理念和运行机制。